

Homework 2 - Solutions

Name: YOUR NAME; Collaborator: YOUR COLLABORATOR

Important: For this homework and all subsequent homeworks, whenever you are asked to analyze a particular algorithm or to design an algorithm, your solution must include a **clear English description** of your thought process. Remember, your audience is a human, not a computer, and we want to understand the process you used to arrive at the solution. This way, we can provide appropriate feedback.

1. You are a backend engineer designing a security feature for a popular web browser. The browser needs to warn users if they attempt to visit a known "malicious" website (e.g., phishing or malware). The database of malicious URLs is massive (containing 10 million entries) and resides on a remote server. To improve speed and privacy, you want to cache a compact version of this database locally on the user's laptop. You decide to use a Bloom Filter for this local cache.

- (a) In the context of blocking malicious websites, a *false positive* occurs when the user attempts to visit a website which the Bloom filter reports as being "malicious," even though it is not in the set of malicious websites. On the other hand, a *false negative* occurs when the user attempts to visit a website which the Bloom filter reports as being "safe" (not in the set of malicious websites), even though it actually is.

- Which type of error will *never* occur and which type may occur in a Bloom filter?

Solution: false negatives never occur in a Bloom filter. If the Bloom filter says "safe" (not in set), it is definitely safe. However, false positives may occur (albeit with a small probability); the Bloom filter may say "malicious," even though the website is safe.

- Which one is merely annoying to the user, and which one a potential security risk?

Solution: False positives are an annoyance to the user - the website will get flagged as potentially unsafe, even though the website is safe (you may have experienced this in your own web browser at times). False negatives, on the other hand, would lead to the user visiting a malicious website while thinking it is safe - this makes the Bloom filter a good candidate for the application, since false negatives never occur!

- (b) Calculate the required bit-array size m (in MB), as well as the number of hash functions k , when the number of items $n = 10,000,000$ and the target error $\epsilon = 0.01$ (i.e., %1).

Solution: Recall the expression for the size of the Bloom filter, where $m \approx n \cdot \ln(1/\epsilon) / \ln^2(2) \approx 2.08n \ln(1/\epsilon)$. In this case, we have the expression

$$m \approx 96,000,000 \text{ bits,}$$

which corresponds to 12 megabytes. The optimal number of hash functions is then set to $k \approx (m/n) \cdot \ln(2)$, which comes out to ~ 6.6 and we can round to $k = 7$.

- (c) Give a comparison for how much smaller the Bloom filter is with respect to the a naive storage of the malicious set of URLs. You may assume that each URL requires 50 bytes to store naively.

Solution: The total size of naively storing a set of 10,000,000 items of 50 bytes each would be:

$$10,000,000 \times 50 \times \frac{1 \text{ mb}}{1,000,000 \text{ bytes}} = 500 \text{ mb},$$

which is about 41 times larger than the size of our 12 mb Bloom filter.

- (d) Six months later, a previously malicious website is cleaned up and is no longer dangerous. It needs to be removed from the blacklist. Can you remove this URL from your standard Bloom Filter? Explain why or why not, referring to how the Bloom filter is constructed.

Solution: One cannot remove elements from a Bloom filter, so we would need to rebuild the Bloom filter. The reason is that certain bits are set if *any of the websites* was hashed to that particular bit. Hence, if we want to remove an element x from the Bloom filter, and we look at the k bits $\mathbf{h}_1(x), \dots, \mathbf{h}_k(x)$; when x was inserted, those bits were all set to 1. However, we do not know whether other elements had also hashed to those bits, so setting any of these bits to 0 could result in false negatives!

2. You are a data engineer at a digital advertising firm, and have just run a massive ad campaign across thousands of different websites and mobile apps. For billing purposes, your client doesn't care how many times the ad was shown overall; they only want to pay for the number of *distinct people* who saw the ad.

Due to strict privacy laws (like GDPR or CCPA), the system could not store raw user ids or IP addresses in the database, so the software engineers implemented a the following architecture: ad impressions fly in and the user id is immediately hashed with k hash functions, inserted into a Bloom Filter of size m , and immediately discarded. The software engineers designed the Bloom filter so as to support a set of size at most n , but the true number of distinct impressions may have been smaller! At the end of the campaign, you are left with a single artifact: the final Bloom Filter bit-array (a long string of 0s and 1s). The client asks: "how many unique people saw the ad?" You know:

- $n \in \mathbb{N}$, which is an *upper bound* on the set of unique impressions.
- The Bloom filter $\mathbf{DS} = \{0, 1\}^m$, and
- $k \in \mathbb{N}$, the number of hash functions used.

Let $n_0 \in \mathbb{N}$ be the true (and unknown) number of unique impressions. Your goal is to reverse-engineer the analysis of the Bloom filter in order to find an estimate of n_0 .

- (a) Let $\mathbf{Z}$ be the random variable given by $\sum_{j=1}^m \mathbf{1}\{\mathbf{DS}[j] = 0\}$. Compute the expectation $\mu = \mathbf{E}[\mathbf{Z}]$ as a function of the true (yet unknown) number of distinct impressions n_0 . Here, we are expecting an expression for μ which depends on m, k and n_0 .

Solution: Here, we use linearity of expectation to say:

$$\mu = \sum_{j=1}^m \Pr[\mathbf{DS}[j] = 0] = m \left(1 - \frac{1}{m}\right)^{kn_0},$$

since $\mathbf{DS}[j] = 0$ if and only if all $k \cdot n_0$ hash evaluations did not map to j .

Notice that, while you do not know n_0 , you can compute $\mathbf{Z}$ from the Bloom filter!

- (b) Use part (a) to “reverse-engineer” an expression, depending on $\mathbf{Z}$, m and k (all quantities that you know!) which gives a potential estimate of n_0 as a function of $\mathbf{Z}$, m and k . The expression should be such that, if $\mathbf{Z}$ is equal to the expectation, we would exactly recover n_0 . (Here, you may want to recall the definition and rules of working with $\ln(\cdot)$).

Solution: Here, we reverse engineer the computation for n_0 .

$$\mu = m \left(1 - \frac{1}{m}\right)^{kn_0} \iff n_0 = \frac{1}{k} \cdot \frac{\ln(m/\mu)}{\ln(m/(m-1))} = \frac{1}{k} \cdot \frac{\ln(\mu/m)}{\ln((m-1)/m)}.$$

Hence, we let the estimator be

$$\frac{1}{k} \cdot \frac{\ln(m/\mathbf{Z})}{\ln(m/(m-1))}.$$

- (c) Use Markov’s inequality to conclude that $\mathbf{Z} \leq 10\mu$ with probability at least 9/10, and as a result, conclude that the estimate you obtain is at least $n_0 - 3m/k$ with probability 9/10.

Solution: Since $\mathbf{Z} \geq 0$, we can use Markov’s inequality to say that $\mathbf{Z} \leq 10\mu$. Hence, if $\mathbf{Y}$ was the estimator,

$$\begin{aligned} \mathbf{Y} &= \frac{1}{k} \cdot \frac{\ln(m/\mathbf{Z})}{\ln(m/(m-1))} \geq \frac{1}{k} \cdot \frac{\ln(m/(10\mu))}{\ln(m/(m-1))} \\ &= \frac{1}{k} \cdot \frac{\ln(m/\mu)}{\ln(m/(m-1))} + \frac{1}{k} \cdot \frac{\ln(1/10)}{\ln(m/(m-1))} \geq n_0 - 3m/k. \end{aligned}$$

3. You are an engineer at a social media platform similar to Twitter (X). The platform processes 100 million tweets per hour. You have been tasked with building a real-time service that tracks the usage frequency of hashtags to identify trending topics. The stream is continuous and the number of possible hashtags enormous, so you cannot store every single hashtag and its count in any standard method. Instead, you choose to use a Count-Min sketch to estimate the frequency of hashtags.

- (a) Give an upper bound on the number of possible hashtags, which we will call n . Assume each hashtag starts with # and is followed by at most 50 characters of the standard English 26-letter alphabet. Hence, we can conceptually imagine a vector $x \in \mathbb{R}^n$ where x_i contains the number of posts with hashtag $i \in [n]$. Give a rough estimate on the number of “yottabytes” (highest used unit of storage) that it would take to explicitly store a count for each hashtag, assuming that we devote 8 bytes for each hashtag.

Solution: Given that there are 50 characters out of an alphabet of 26 which specify the hashtag, there are a total of 26^{50} possible hashtags. If one devotes 8 bytes to each hashtag, the total memory consumption becomes:

$$26^{50} \times 8 \text{ bytes} \times \frac{1 \text{ yottabytes}}{10^{24} \text{ bytes}} \approx 10^{47} \text{ yottabytes}.$$

- (b) The Count-Min sketch will provide an approximate count, which means that for any particular hashtag $i \in [n]$, it will output a number $\hat{x}_i \in \mathbb{R}$. Decide which of the following is always true, and which may or may not be true:

- $\hat{x}_i \leq x_i$ for all $i \in [n]$.
- $\hat{x}_i \geq x_i$ for all $i \in [n]$.

Give a brief justification for each.

Solution: In the Count-Min sketch, we always report the minimum number seen in the hash buckets, which guarantees that $\hat{x}_i \geq x_i$ for all $i \in [n]$ always. The case $\hat{x}_i \leq x_i$ may occur only if there were few elements inserted such that there are no hash collisions in some hash bucket.

- (c) For an accuracy parameter of $\epsilon = 0.01$, use Markov's inequality to determine the value of m , such that a single-hash Count-Min sketch of size m can estimate, for any fixed hashtag $i \in [n]$, the value of x_i up to an additive error of $\epsilon\|x\|_1$ with probability at least 0.9.

Solution: If we use a single-hash Count-Min sketch, denoted by the array DS, then we will report as our estimate

$$\hat{x}_i = \text{DS}[\mathbf{h}(i)] = x_i + \sum_{j \in [d] \setminus \{i\}} \mathbf{1}\{\mathbf{h}(j) = \mathbf{h}(i)\} \cdot x_j,$$

which means that the error $\hat{x}_i - x_i$ is a positive random variable with expectation $\sum_{j \in [d] \setminus \{i\}} x_j / m = \|x\|_1 / m$. If we let $m = 10/\epsilon$, then the expected error becomes $\epsilon\|x\|_1/10$, so by Markov's inequality, the error is at most $\epsilon\|x\|_1$ with probability at least 0.9.

- (d) Using the median trick, specify the number of times ℓ to repeat the single-hash Count-Min sketch such that for all $i \in [n]$, x_i may be estimated up to additive error $\epsilon\|x\|_1$ with probability 0.9.

Solution: For any fixed index $i \in [n]$, in order for the median of ℓ repetitions to be incorrect, there must be at least $\ell/2$ of the instances which are incorrect (and each is incorrect with probability at most 0.1). Hence, the probability that there exists an index $i \in [n]$ such that at least $\ell/2$ instance is at most

$$n \cdot \binom{\ell}{\ell/2} 0.1^{\ell/2} \leq n \cdot (4 \cdot 0.1)^{\ell/2} \leq \frac{n}{2^{\ell/2}},$$

which means it suffices to set $\ell = 2 \log_2(10n)$ for the above probability to be at most 0.1.

- (e) Based on your answers to (c) and (d), give an estimate for the total memory in megabytes required, assuming we devote 8 bytes to store numbers.

Solution: We have to maintain $2 \log_2(10n)$ arrays of size $10/\epsilon$, and each using 8 bytes. Hence, the total becomes:

$$\begin{aligned} 2 \log_2(10n) \times \frac{10}{\epsilon} \times 8 \text{ bytes} &= 2 \log_2(10 \cdot 26^{50}) \times 10 \times 100 \times 8 \text{ bytes} = 3.8 \times 10^6 \text{ bytes} \\ &= 3.8 \text{ megabytes.} \end{aligned}$$

Your platform scales up, and you now have 3 separate servers ingesting tweets to handle the load. Each server maintains its own local Count-Min sketch which uses the *same* hash functions into m positions with ℓ repetitions.

- (f) How can you combine these three local sketches into a single global sketch that represents the total counts across all servers?

Solution: Each sketch consists of ℓ arrays of m positions, hence, we can represent this as a 2D array $\ell \times m$. We simply add the three sketches up.

- (g) Give a brief justification for why it is important that all three servers use the same hash functions.

Solution: It is important that the three servers use the same randomness so that the hash positions will go to a consistent bucket. This means that a tweet `#cats` will map to the same hash buckets, irrespective of the server that tweet was processed.